

实践7：使用 GMR 完成人体动作到 G1 的运动重定向



深蓝学院《人形机器人运动控制》课程第5章

课程链接：<https://www.shenlanxueyuan.com/course/864>

1. 作业目标

💡 实践 7（GMR）与实践 8（AMP）为前后衔接的关联作业：实践 7 用 GMR 生成的重定向运动数据，需要接入实践 8 AMP 算法训练。

实践7 基于开源工具 [GMR \(General Motion Retargeting\)](#)，完成人体动作数据到 Unitree G1 29DoF 人形机器人的运动重定向，核心流程如下：

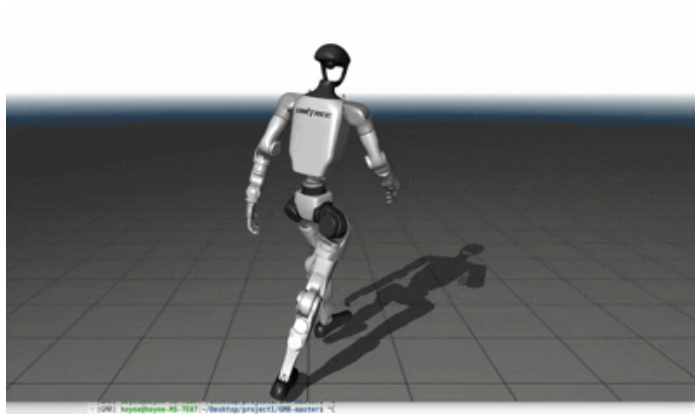
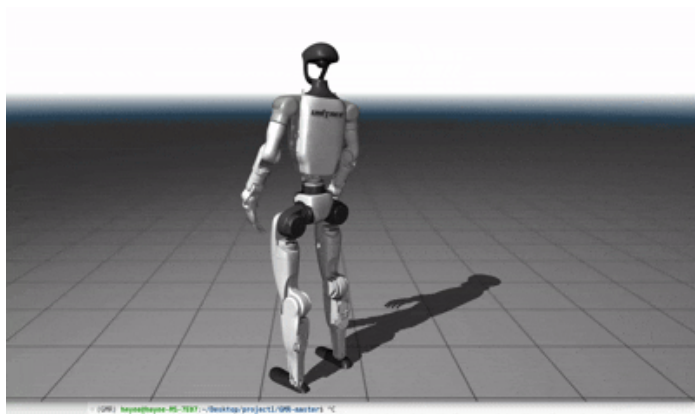
代码块

```
1  AMASS/ACCAD SMPL-X Motion
2      ↓
3  Human Body Motion
4      ↓
5  GMR Motion Retargeting
6      ↓
7  Unitree G1 Reference Motion
8      ↓
9  AMP NPZ Dataset
```

你需要修改源码中的 `smp1x_to_robot_dataset.py` 或者 `smp1x_to_robot.py`，或新建代码文件 `smp1x_to_robot_dataset_npz.py` 和 `smp1x_to_robot_npz.py`，实现两项核心要求：

- 重定向结果可在 GMR 自带的 MuJoCo 窗口中可视化播放；
- 输出符合规范的 `.npz` 格式数据，可被实践 8 的 AMP 数据加载器直接读取与训练。

我们提供的参考样例 `B12_-_walk_turn_right_(90)_stageii.npz` 用于说明字段、维度和坐标约定。你需要自行生成新的走路、跑步、走跑切换或转弯等全新动作数据。



2. 能力目标

希望通过完成本次作业，你能具备以下能力：

能力	要求
理解运动重定向	说明人体骨架与机器人骨架之间的结构差异
使用 AMASS 数据	能读取 SMPL-X 格式的人体动作
配置 GMR	能选择 unitree_g1 并运行重定向
理解关键身体映射	能解释人体关键点如何对应到 G1 连杆
理解坐标和缩放	能处理坐标系、人体身高和局部缩放
检查 IK 结果	能发现穿模、滑脚、关节越界等问题
输出 AMP 数据	能生成符合指定字段和形状的 .npz
对接 实践 8	输出数据可由 motion_dataset.py 直接加载

3. 作业理论背景

3.1 GMR 动作重定向

GMR 是一款通用运动重定向工具，以人体关键身体部位的位置与旋转为目标，通过逆运动学（IK）求解机器人根节点位姿与关节角度。

GMR 官方仓库：<https://github.com/YanjieZe/GMR>

由于人体与机器人存在天然结构差异（人体腰部、肩部和脚部自由度与 G1 不完全一致；人体手臂和腿长比例与机器人不同；人体动作可能超出 G1 的关节范围；人体脚掌接触方式与机器人脚掌结构不

同。），运动重定向无法直接复制关节角度，需完成完整的适配流程：

1. 坐标系对齐；
2. 零位姿态（rest pose）对齐；
3. 人体与机器人局部尺度调整；
4. 人体关键身体到机器人连杆的映射；
5. 位置和旋转约束下的 IK 求解；
6. 关节角度范围和速度限制约束；
7. 足部接触及动作质量检查。

GMR 官方原生支持从 SMPL-X 格式的 AMASS 数据到 Unitree G1 的重定向，并将机器人单帧运动表示为根节点平移、根节点旋转和机器人关节位置三类核心数据。

3.2 AMASS/ACCAD 数据集

AMASS（Archive of Motion Capture as Surface Shapes）将多套人体动作捕捉数据集统一为参数化人体模型表示，适配批量运动重定向场景。本次作业推荐使用 AMASS 数据集中的 ACCAD 子数据集。

AMASS 官方网站：<https://amass.is.tue.mpg.de/>

数据下载要求：

- 路径选择：AMASS → SMPL-X → ACCAD
- 不要下载 SMPL+H 版本，需遵循 GMR 官方数据准备规范使用 SMPL-X 格式数据，[GMR 数据准备说明](#)



我们提前下载了ACCAD数据，约1.04G，存放在百度网盘：

<https://pan.baidu.com/s/1uOQoYtDs6-hwgDHBjMgpag?pwd=kh5i>

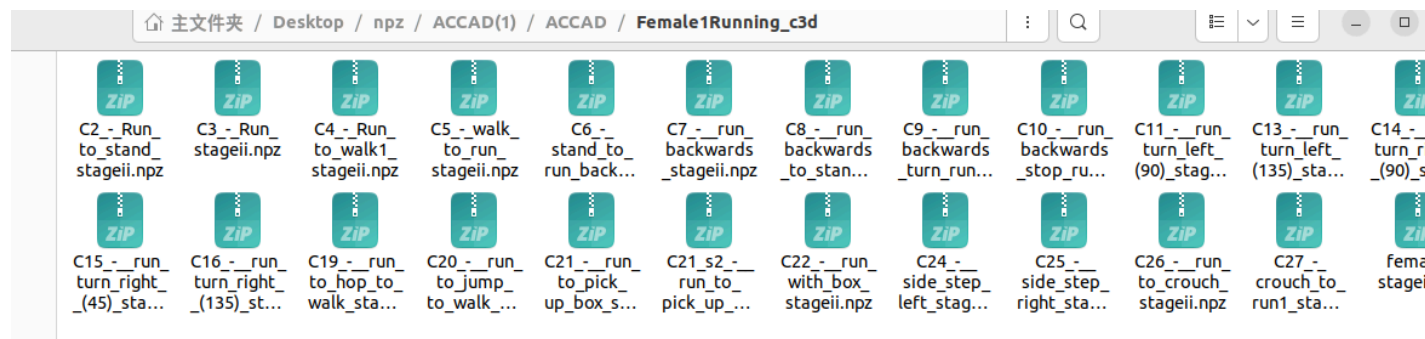
动作选择建议：

优先选择基础 locomotion 类动作，包括：

- 站立到行走；
- 稳定向前行走；
- 向后行走；
- 左右转弯；
- 原地转身；
- 慢跑或向前跑；
- 站立到跑步；

- 行走到跑步；
- 跑步到停止。

可以优先查看 ACCAD 中与 `General`、`Running`、`WalksTurns` 等相关的动作目录。**不建议在初始阶段选择翻滚、爬行、躺地、跳跃或复杂武术动作**，此类动作对 IK 和接触处理要求更高，易出现重定向失败。



 你可以从数据文件名判断动作类型。

4. 环境与文件配置

4.1 基础运行环境

下载并解压GMR源码后，进入源码根目录：

代码块

```
1 cd /path/to/GMR-master # 注意替换为你的本地实际路径
```

创建并激活 Conda 环境：

代码块

```
1 conda create -n gmr python=3.10 -y
2 conda activate gmr
3
```

注意：在安装依赖前，需修改GMR-master文件夹中 `setup.py` 文件，以固定部分依赖的版本，修改如下：

代码块

```
1 "numpy==1.26.4",
```

```
2 "scipy==1.15.3",
```

然后按照安装所有依赖：

代码块

```
1 pip install -e .
```

如果已经创建过环境或者安装过GMR，仅需执行激活与路径切换：

代码块

```
1 conda activate gmr
2 cd /path/to/GMR-master # 注意替换为你的本地实际路径
```

4.2 作业核心文件

GMR 源码中与本次作业相关的文件结构如下：

代码块

```
1 GMR-master/
2 |— general_motion_retargeting/
3 |   |— motion_retarget.py      # GMR核心IK求解逻辑，实现人体目标到机器人关节的转
   换
4 |   |— kinematics_model.py     # 机器人正运动学模块，用于计算local_body_pos与连
   杆列表
5 |   |— robot_motion_viewer.py  # 运动可视化模块
6 |   |— params.py
7 |   |— utils/
8 |     |— smpl.py
9 |     |— ik_configs/
10 |       |— smplx_to_g1.json    # SMPL-X与G1的身体映射、权重与缩放配置
11 |
12 |— scripts/
13 |   |— smplx_to_robot.py        # 单条动作重定向基础脚本
14 |   |— smplx_to_robot_dataset.py # 批量动作重定向基础脚本
15 |   |— smplx_to_robot_npz.py    # 需要你新建的代码文件（处理单个文件）
16 |   |— smplx_to_robot_dataset_npz.py # 需要你新建的代码文件（处理多个文件）
17 |   |— vis_robot_motion.py     # 本地运动文件可视化脚本
18 |   |— vis_robot_motion_npz.py  # 重定向结果可视化脚本（随作业提供）
19 |
20 |— assets/
21 |   |— body_models/smplx/
22 |   |— unitree_g1/
```

motion_retarget.py

实现 GMR 的主要 IK 求解过程，将人体关键身体目标转换为机器人关节状态。

kinematics_model.py

提供机器人正运动学。输入 G1 根节点位姿和关节角，计算各机器人连杆的位置与旋转。作业中使用它来计算 `local_body_pos` 和 `link_body_list`。

smplx_to_g1.json

定义 SMPL-X 人体身体部位与 G1 连杆的对应关系，以及不同目标的权重和缩放设置。

smplx_to_robot.py

用于单条动作重定向和可视化。需注意，GMR 开源版本仅导出 `fps`、`root_pos`、`root_rot`、`dof_pos` 四个字段，缺少 AMP 训练必需的 `local_body_pos` 与 `link_body_list`，无法直接用于实践 8。

需要你新建并修改的代码文件： `smplx_to_robot_npz.py`

用于处理单个 SMPL-X 动作，并计算正运动学结果。

需要你新建并修改的代码文件： `smplx_to_robot_dataset_npz.py`

用于批量处理 SMPL-X 动作，并计算正运动学结果。

4.3 人体模型文件

SMPL-X 人体模型是人体的基础骨架/网格模板，定义了人体的关节结构、骨骼拓扑、网格形态，是 GMR 做运动重定向的基准人体骨架，相当于“人体的标准模板”。

我们提供了3个 `.pkl` 文件，请放在 GMR 源码的指定目录 `GMR-master/assets/body_models/smplx/` 路径下。

 SMPL-X人体模型模型文件链接: https://pan.baidu.com/s/1Luax709F_KAIPf5rVUK0Ow?pwd=7mq2

代码块

```
1  GMR-master/
2  └── assets/
3      └── body_models/
4          └── smplx/
```

```
5      └─ SMPLX_NEUTRAL.pkl
6      └─ SMPLX_FEMALE.pkl
7      └─ SMPLX_MALE.pkl
```

5. 操作流程

5.1 单条动作预览验证

先通过单条动作验证环境、数据与重定向逻辑是否正常：

代码块

```
1 python scripts/smplx_to_robot_npz.py \
2     --smplx_file /path/to/ACCAD/example_stageii.npz \
3     --robot unitree_g1 \
4     --save_path /tmp/example.npz \
5     --rate_limit
```

注意：该命令仅用于效果预览，生成的 npz 文件字段不完整，不可作为最终提交数据。

5.2 批量生成完整 AMP 数据

确认单条动作重定向正常后，使用批量脚本生成完整格式的训练数据。

- 单进程测试（显存不足时推荐，避免多进程占用 GPU）：

代码块

```
1 python scripts/smplx_to_robot_dataset_npz.py \
2     --src_folder /path/to/AMASS/ACCAD \
3     --tgt_folder /path/to/output/g1_amp_npz \
4     --robot unitree_g1 \
5     --num_cpus 1
```

- 多进程批量生成，确认一条动作能够正常生成后，再增加并行进程：

代码块

```
1 python scripts/smplx_to_robot_dataset_npz.py \
2     --src_folder /path/to/AMASS/ACCAD \
3     --tgt_folder /path/to/output/g1_amp_npz \
4     --robot unitree_g1 \
5     --num_cpus 4 \
```

参数说明： `--override` 参数会覆盖目标路径下已存在的同名文件。

脚本内部使用 `cuda:0` 计算正运动学。

显存不足时应使用 `--num_cpus 1`，避免多个进程同时占用 GPU。

5.4 可视化数据

我们提供了将重定向结果可视化的脚本，命令如下：

代码块

```
1 python scripts/vis_robot_motion_npz.py \
2   --robot unitree_g1 \
3   --robot_motion_path "/tmp/example.npz" # 请注意修改为自己的npz文件
```

6. 输出数据格式规范

6.1 基础读取要求

请确认 `.npz` 必须能够使用以下方式读取，不能存为 Python 对象、字典对象或 `None`。

代码块

```
1 with np.load(file_path, allow_pickle=False) as data:
2     ...
```

6.2 必选字段说明

每个 `.npz` 文件必须包含以下 6 个字段，维度与定义严格匹配：

字段	形状	含义
fps	标量	动作帧率，必须大于 0
root_pos	(T, 3)	G1 根节点世界坐标位置
root_rot	(T, 4)	G1 根节点四元数，顺序为 xyzw
dof_pos	(T, 29)	G1 29 个关节的位置，单位为弧度
local_body_pos	(T, N, 3)	各 G1 连杆在根节点坐标系中的位置

link_body_list	(N,)	local_body_pos 第二维对应的连杆名称
----------------	------	---------------------------

其中：

- T 是动作总帧数；
- N 是保存的机器人连杆数量；
- 所有时序字段的第一维必须相同；
- 数值中不能包含 NaN 或 Inf。

样例文件的实际结构参考：

代码块

```

1  fps                ()                float64
2  root_pos           (186, 3)          float64
3  root_rot           (186, 4)          float64
4  dof_pos            (186, 29)         float64
5  local_body_pos     (186, 38, 3)      float32
6  link_body_list     (38,)             Unicode string

```

数值数据可以使用 float32 或 float64 ，建议统一保存为 float32 以减少文件体积。

6.3 坐标系和单位约定

root_pos

- `root_pos[t] = [x, y, z]`
- 使用世界坐标系；
- 长度单位为米；
- z 轴向上；
- z 高度应使机器人脚部接近地面；
- 建议将第一帧的 x、y 平移到原点，即： $p_{xy}(t) \leftarrow p_{xy}(t) - p_{xy}(0)$
- 不能把 root_pos 保存为人体 SMPL-X 的 pelvis 坐标，必须保存 GMR 求解后的 G1 根节点位置。

root_rot

- 要求使用 `[x, y, z, w]` ，也就是 xyzw 顺序；
- GMR 内部用于 MuJoCo 可视化的 qpos 四元数通常为 `[w, x, y, z]` （wxyz）顺序，保存前必须完成重排：

```
代码块 root_rot_xyzw = root_rot_wxyz[:, [1, 2, 3, 0]]
```

- 需校验四元数模长满足： $\|q_t\|_2 \approx 1$ ；
- 四元数顺序错误会导致机器人整体倒置、旋转异常或姿态跳变。

dof_pos

- 维度固定为 `shape = (T, 29)`；
- 单位为弧度 `unit = rad`，不能使用角度制；
- 不能直接保存 SMPL-X 人体关节参数。

6.4 G1 29DoF 关节顺序

如果文件中不额外保存 `joint_names`，**实践 8** 会按照下面的固定顺序解释 `dof_pos`：

代码块

```
1  1. left_hip_pitch_joint
2  2. left_hip_roll_joint
3  3. left_hip_yaw_joint
4  4. left_knee_joint
5  5. left_ankle_pitch_joint
6  6. left_ankle_roll_joint
7  7. right_hip_pitch_joint
8  8. right_hip_roll_joint
9  9. right_hip_yaw_joint
10 10. right_knee_joint
11 11. right_ankle_pitch_joint
12 12. right_ankle_roll_joint
13 13. waist_yaw_joint
14 14. waist_roll_joint
15 15. waist_pitch_joint
16 16. left_shoulder_pitch_joint
17 17. left_shoulder_roll_joint
18 18. left_shoulder_yaw_joint
19 19. left_elbow_joint
20 20. left_wrist_roll_joint
21 21. left_wrist_pitch_joint
22 22. left_wrist_yaw_joint
23 23. right_shoulder_pitch_joint
24 24. right_shoulder_roll_joint
25 25. right_shoulder_yaw_joint
26 26. right_elbow_joint
27 27. right_wrist_roll_joint
```

```
28 28. right_wrist_pitch_joint
29 29. right_wrist_yaw_joint
```

如果 `dof_pos` 的列顺序错误，动作就会映射到错误关节。例如髋关节角可能被发送到肩关节，数据虽然能读取，但完全不能使用。

6.5 `local_body_pos` 的定义

`local_body_pos` 不是世界坐标中的连杆位置，而是各连杆相对机器人根节点的位置，计算公式：

$$p_i^b = R_b^{-1}(p_i^w - p_b^w)$$

其中：

- p_i^w 为连杆世界坐标位置；
- p_b^w 为机器人根节点世界位置；
- R_b 为根节点旋转；
- p_i^b 为根节点坐标系中的连杆位置。

推荐你在修改 `smplx_to_robot_dataset_npz.py` 的时候，使用以下思路计算该字段：

代码块

```
1 root position = [0, 0, 0]
2 root rotation = identity
3 joint position = dof_pos
4           ↓
5   G1 forward kinematics
6           ↓
7       local_body_pos
```

不能直接把人体关键点、SMPL-X joints 或世界坐标中的机器人连杆位置保存为 `local_body_pos`。

6.6 AMP 必需的关键连杆

实践 8 默认使用以下 4 个关键连杆：

代码块

```
1 left_ankle_roll_link
2 right_ankle_roll_link
3 left_wrist_yaw_link
```

```
4 right_wrist_yaw_link
```

因此 `link_body_list` 至少必须包含这四个名称。

允许保存完整 G1 连杆列表，例如样例中的 38 个连杆：

代码块

```
1 local_body_pos.shape = (T, 38, 3)
2 link_body_list.shape = (38,)
```

AMP 数据加载器会根据名称自动选择所需的四个连杆。

如果缺少任意一个关键连杆，**实践 8** 会报错：

代码块

```
1 Motion file is missing configured key links
```

6.7 不需要保存的字段

最终 `.npz` 文件不要求保存以下字段：

代码块

```
1 joint_vel
2 root_lin_vel
3 root_ang_vel
4 projected_gravity
5 motion_command
```

实践 8 的 `motion_dataset.py` 会根据相邻帧自动计算：

- 关节速度；
- 根节点线速度；
- 根节点角速度；
- 投影重力；
- 对应的速度命令。

因此，**实践 7** 最重要的是保证位置、姿态、帧率和时间连续性正确。

6.8 数据保存方式

推荐保存方式：

代码块

```
1  np.savez_compressed(  
2      output_path,  
3      fps=np.asarray(fps, dtype=np.float64),  
4      root_pos=np.asarray(root_pos, dtype=np.float32),  
5      root_rot=np.asarray(root_rot, dtype=np.float32),  
6      dof_pos=np.asarray(dof_pos, dtype=np.float32),  
7      local_body_pos=np.asarray(local_body_pos, dtype=np.float32),  
8      link_body_list=np.asarray(link_body_list, dtype=str),  
9  )
```

禁止使用：

代码块

```
1  np.array(None)  
2  dtype=object  
3  allow_pickle=True
```

虽然 object 数据可以通过 pickle 读取，但**实践 8** 为避免不安全的对象反序列化，固定使用：

代码块

```
1  np.load(path, allow_pickle=False)
```

不符合要求的文件将无法加载。

6.1 .npz 格式检查

生成文件后，可运行以下脚本完成格式自动化校验：

代码块

```
1  python - <<'PY'  
2  import numpy as np  
3  
4  path = "/path/to/output_motion.npz"  
5  
6  required = {
```

```

7     "fps",
8     "root_pos",
9     "root_rot",
10    "dof_pos",
11    "local_body_pos",
12    "link_body_list",
13 }
14
15 with np.load(path, allow_pickle=False) as data:
16     print("keys:", data.files)
17
18     missing = required - set(data.files)
19     assert not missing, f"missing fields: {sorted(missing)}"
20
21     fps = float(np.asarray(data["fps"]).item())
22     root_pos = data["root_pos"]
23     root_rot = data["root_rot"]
24     dof_pos = data["dof_pos"]
25     local_body_pos = data["local_body_pos"]
26     links = data["link_body_list"].astype(str)
27
28     assert fps > 0
29     assert root_pos.ndim == 2 and root_pos.shape[1] == 3
30     assert root_rot.shape == (root_pos.shape[0], 4)
31     assert dof_pos.shape == (root_pos.shape[0], 29)
32     assert local_body_pos.ndim == 3
33     assert local_body_pos.shape[0] == root_pos.shape[0]
34     assert local_body_pos.shape[2] == 3
35     assert local_body_pos.shape[1] == len(links)
36
37     for array in (root_pos, root_rot, dof_pos, local_body_pos):
38         assert np.isfinite(array).all()
39
40     quat_norm = np.linalg.norm(root_rot, axis=1)
41     assert np.max(np.abs(quat_norm - 1.0)) < 1e-3
42
43     required_links = {
44         "left_ankle_roll_link",
45         "right_ankle_roll_link",
46         "left_wrist_yaw_link",
47         "right_wrist_yaw_link",
48     }
49     assert required_links.issubset(set(links))
50
51     print("fps:", fps)
52     print("root_pos:", root_pos.shape)
53     print("root_rot:", root_rot.shape)

```

```
54     print("dof_pos:", dof_pos.shape)
55     print("local_body_pos:", local_body_pos.shape)
56     print("NPZ format check passed.")
57 PY
```

校验通过输出参考：

代码块

```
1  keys: ['fps', 'root_pos', 'root_rot', 'dof_pos', 'local_body_pos',
        'link_body_list']
2  fps: 29.903329752953812
3  root_pos: (232, 3)
4  root_rot: (232, 4)
5  dof_pos: (232, 29)
6  local_body_pos: (232, 38, 3)
7  link_body_list: (38,)
8  NPZ format check passed.
```

6.2 动作质量检查

每条动作至少检查以下内容：

检查项	合格标准
整体方向	机器人朝向与人体运动方向一致
根节点高度	双脚接近地面，身体不过度悬空或下沉
足部滑动	支撑脚阶段不应持续明显移动
左右腿映射	左右侧不能颠倒
手臂映射	肩、肘、腕运动方向合理
关节范围	无明显超出 G1 关节范围
姿态连续性	相邻帧不出现突然翻转
四元数	模长接近 1，无符号跳变造成的插值异常
动作速度	播放速度与原人体动作基本一致
自碰撞	手臂不应长期穿过躯干或双手严重重叠

7. 常见问题

1. 生成的 NPZ 缺少 `local_body_pos`

原因：使用了当前的 `smplx_to_robot_npz.py` 单条脚本。

处理：使用 `smplx_to_robot_dataset_npz.py`，或者在单条脚本中加入 G1 正运动学计算。

2. `Object arrays cannot be loaded`

原因：保存了 `None`、Python list object 或其他 `dtype=object` 数据。

处理：删除 object 字段，并将 `link_body_list` 转换为 Unicode NumPy 数组。

3. 机器人整体倒置

原因：把 GMR 的 `wxyz` 四元数直接当作 `xyzw` 保存。

处理：保存前按照 `[1, 2, 3, 0]` 重新排列。

4. 机器人动作左右错乱

原因：`dof_pos` 的关节顺序与 G1 29DoF 顺序不一致。

处理：按照本作业给出的 29 个关节顺序重新排列。

5. AMP 报缺少关键连杆

原因：`link_body_list` 中缺少脚踝或手腕名称。

处理：使用完整 G1 正运动学 body list，并确认名称完全一致。

6. 数据能加载但机器人滑脚严重

说明文件格式正确，但重定向质量不合格。需要重新检查根节点移动、身体缩放、足端约束和 IK 权重。

8. 作业提交内容

8.1 提交内容清单

- **代码文件：**修改/新建的 NPZ 导出脚本源码
- **数据文件：**至少包含 3 条符合规范的动作数据
 - 1 条走路动作；
 - 1 条跑步动作；
 - 1 条走跑切换或转弯动作；
 - 每条动作均为规定的 `.npz` 格式。

- 可视化视频：G1 重定向动作的录制视频，需清晰展示脚部接触、身体朝向、手臂动作和走路或跑步节奏。

8.2 提交前检查

- 使用的是 AMASS SMPL-X 数据，而不是 SMPL+H；
- GMR 目标机器人为 `unitree_g1`；
- 输出文件扩展名为 `.npz`；
- 文件能够用 `allow_pickle=False` 读取；
- 包含全部 6 个必需字段；
- `dof_pos.shape == (T, 29)`；
- `root_rot` 使用 `xyzw`；
- `local_body_pos` 是根节点局部坐标；
- `link_body_list` 包含双脚踝和双手腕关键连杆；
- 所有时间序列具有相同帧数；
- 数据中没有 `NaN`、`Inf` 或 `object`；
- 动作没有明显穿模、滑脚或关节越界；
- 数据已放入实践 8 的对应 `motion` 目录；
- `motion_cfg.py` 的文件名和采样权重已经同步。

8.3 评分标准

项目	分值
GMR 环境配置与 Unitree G1 重定向流程跑通	20
AMASS/ACCAD 数据选择与人体动作读取	10
人体到 G1 的关键身体映射与尺度处理	15
IK 结果、足部接触和关节范围合理	20
NPZ 字段、维度、坐标和四元数格式正确	20
数据能够被实践 8 的 AMP 加载器直接读取	5
可视化对比和实验报告	10

总分	100
----	-----
